DOKUMEN EVALUASI PRAKTIKAL (ADVANCED ASSESSMENT) PROGRAM: CYBERSECURITY ETHICAL HACKER

MENTOR UTAMA: Bagus Sudung Prakoso

Kelompok 8 :
- Afnan Yazid Pradana
- Kelvin Febrianto
- Kurniawan Saputra
- Rishfan Mailizar
- Robin Felix Hama
Cybersecurity Ethical Hacker

PENYELENGGARA: PT VINIX SEVEN AURUM

Topik: OWASP Top 10 - Vulnerability Chaining, Filter Bypassing & Secure Coding **Target Sistem:** Portal Helpdesk Internal - PT Teknologi Aman Sejahtera **Tautan Lab:** `https://seclab-week-6--bagussudung1602.replit.app`

A. DESKRIPSI TUGAS Anda diberikan akses ke sistem *Helpdesk Ticketing* internal perusahaan. Tim *developer* mengklaim bahwa aplikasi ini sudah "aman" karena mereka telah merancang *Custom Blacklist Filter* pada *form login* dan input pembuatan tiket. Tugas Anda sebagai *Ethical Hacker* adalah membuktikan bahwa filter berbasis *blacklist* statis tersebut cacat. Anda ditugaskan untuk melakukan *bypass*, merangkai eksploitasi (*vulnerability chaining*) dari *Stored XSS* hingga *Account Takeover*, dan pada akhirnya memberikan rancangan perbaikan di level kode (*Secure Coding*).

B. PETUNJUK PENGERJAAN

1. **Logika di atas Tooling:** Aplikasi ini didesain menyerupai *production environment* tanpa petunjuk (*hint*). Filter *blacklist* akan secara otomatis memblokir *payload* generik dari internet. DILARANG keras menggunakan *Automated Scanner* (seperti Active Scan ZAP/Burp). Lakukan inspeksi dan perakitan *payload evasion* secara manual.
2. Tangkapan layar (*screenshot*) wajib menampilkan bukti keberhasilan eksploitasi dan modifikasi *request* pada *Burp Suite Repeater* atau *Browser DevTools*.

C. INSTRUKSI DAN RINCIAN TUGAS

BAGIAN 1: AUTHENTICATION BYPASS & FILTER EVASION (SQLi) - Bobot: 30% Tim pengembang mencoba mencegah *SQL Injection* dengan memblokir penggunaan karakter Spasi, string `OR` (*case-insensitive*), dan logika `1=1` pada kolom *Username* di endpoint `/login`.

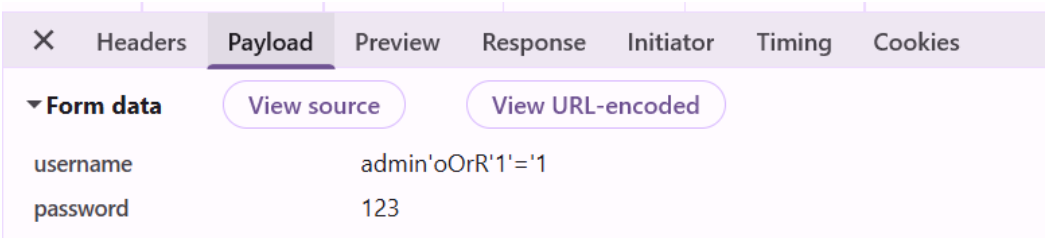
• *Asumsi Snippet Kode Filter (Whitebox):*

Python

```
username = request.form['username']
username = username.replace(" ", "").replace("OR", "").replace("or",
"").replace("1=1", "")
query = f"SELECT * FROM users WHERE username='{username}' AND
password='{password}'"
```

1. **The Bypass:** Rancang *payload SQL Injection* khusus yang memanipulasi kolom *Username* TANPA menggunakan karakter spasi, TANPA kata `OR/or`, dan TANPA `1=1`. *(Petunjuk: Eksplorasi penggunaan komentar SQL `/**/` sebagai pengganti spasi dan operator logika lain seperti `||`).*

Jawaban :

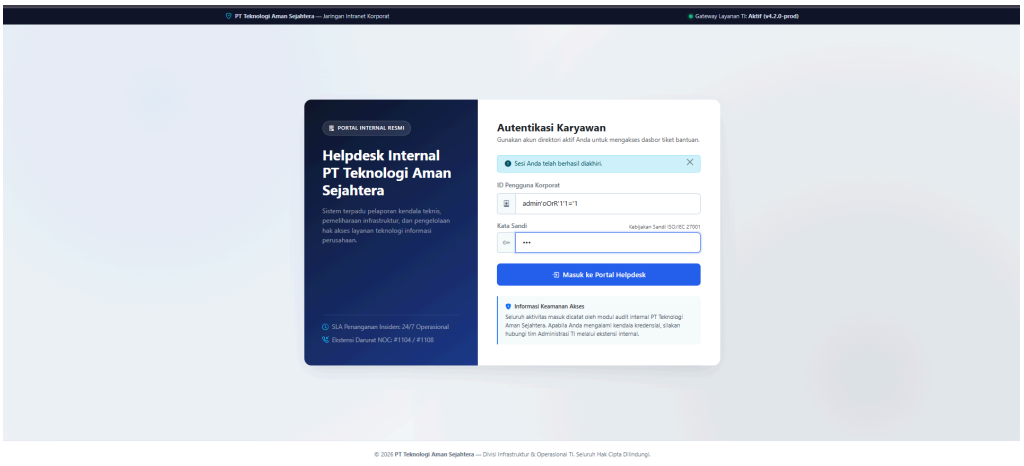


Gambar 1.menunjukan payload yang digunakan

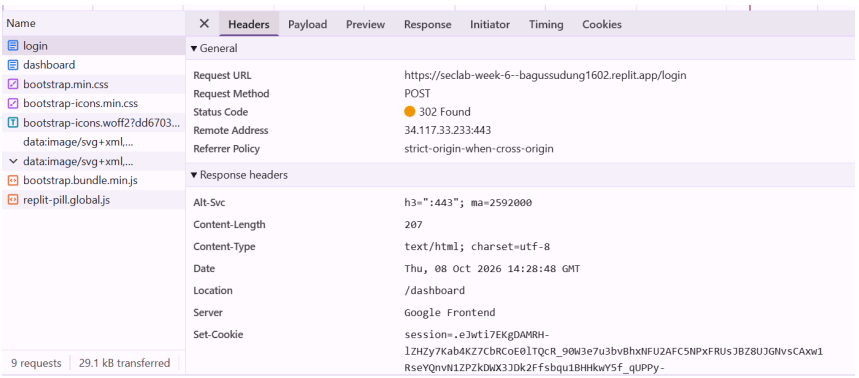
Payload yang digunakan adalah: admin'oOrR'1'='1 (sebagai username)

2. **Dokumentasi:** Lampirkan *payload* akhir yang berhasil membuat Anda masuk ke dasbor dan jelaskan secara teknis mengapa filter `replace()` berantai yang dibuat *developer* tersebut gagal mengamankan *query*.

Jawaban :



Gambar 2.menunjukan form login yang diisi dengan payload admin'oOrR'1'='1



Gambar 3.menunjukan hasil response pada request api login

Pengujian terhadap endpoint /login berhasil menunjukkan bahwa mekanisme Custom Blacklist Filter dapat dilewati menggunakan payload :

admin'oOrR'1'='1

Payload tersebut berhasil digunakan untuk memperoleh akses ke dashboard tanpa mengetahui kata sandi akun target.Filter yang diterapkan oleh aplikasi hanya melakukan penghapusan terhadap karakter spasi dan string OR, or, serta 1=1 menggunakan fungsi replace(). Pendekatan tersebut bersifat blacklist dan tidak melakukan normalisasi input maupun menggunakan parameterized query.

Pada payload yang digunakan, string oOrR tidak cocok secara literal dengan substring OR maupun or, sehingga tidak dihapus oleh filter. Selain itu, ekspresi '1'='1' merupakan bentuk kondisi Boolean yang secara semantik setara dengan kondisi selalu benar, tetapi tidak sama dengan string 1=1 yang secara eksplisit diblokir oleh filter.

Keberhasilan payload membuktikan bahwa filtering berbasis blacklist tidak dapat digunakan sebagai mekanisme utama untuk mencegah SQL Injection. Penyerang dapat memanfaatkan variasi sintaks, representasi karakter, operator, maupun ekspresi ekuivalen yang tidak tercakup dalam daftar blacklist.

Root cause vulnerability terdapat pada konstruksi query menggunakan string concatenation :

```
query = f"SELECT * FROM users WHERE username='{username}' AND password='{password}'"
```

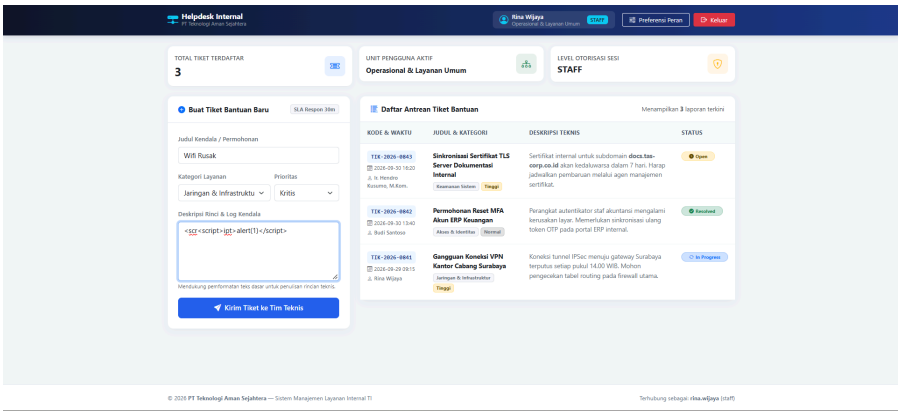
Input pengguna secara langsung digabungkan dengan SQL statement sehingga karakter kontrol SQL dari input pengguna dapat mengubah struktur query.Perbaikan yang direkomendasikan adalah mengganti string concatenation dengan parameterized query/prepared statement. Selain itu, password sebaiknya disimpan menggunakan password hashing yang aman seperti Argon2id atau bcrypt dan diverifikasi menggunakan fungsi password verification yang sesuai.

Dengan demikian, pengujian berhasil membuktikan bahwa Custom Blacklist Filter tidak memberikan perlindungan yang memadai terhadap SQL Injection dan dapat menyebabkan authentication bypass

BAGIAN 2: VULNERABILITY CHAINING (STORED XSS to CSRF) - Bobot: 40% Di dalam Dasbor, Anda menemukan formulir pembuatan Tiket Helpdesk. Untuk mencegah XSS, *developer* telah membuat fungsi sanitasi yang secara TEPAT menghapus *string* `<script>`, `onload`, `onerror`, dan `javascript:` menjadi *string* kosong `""`. Namun, celahnya adalah penghapusan ini hanya dilakukan **satu kali iterasi** (tidak rekursif).

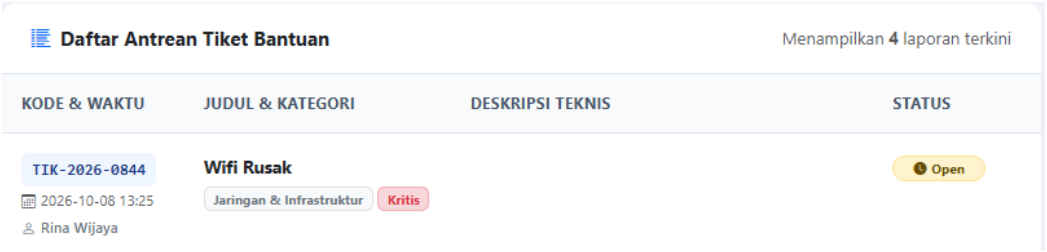
1. **XSS Filter Evasion:** Buat *payload Stored XSS* pada deskripsi tiket yang dapat menembus filter iterasi tunggal tersebut. Anda bisa menggunakan teknik *Tag Nesting* (contoh: `<scr<script>ipt>`) atau menggunakan *Event Handler* HTML5 langka. Buktikan keberhasilan eksekusi XSS dengan memunculkan *pop-up* sederhana.

jawaban :



Gambar 1.Buat Tiket Bantuan Baru

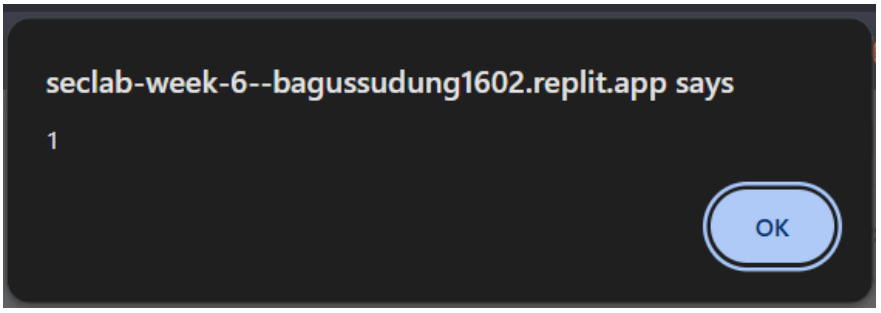
Pada gambar 1terlihat formulir “Buat Tiket Bantuan Baru” pada Portal Helpdesk Internal. Penguji mengisi data tiket dan memasukkan payload pada bagian Deskripsi Risiko & Log Kendala. Payload yang digunakan memanfaatkan teknik Tag Nesting, yaitu <scr<script>ipt>alert(1)</script>. Teknik ini digunakan untuk melewati filter XSS yang hanya melakukan proses penghapusan satu kali. Setelah seluruh data diisi, tiket dikirim dengan menekan tombol Kirim Tiket ke Tim Teknis.



Gambar 2.Menunjukan Daftar Antrean Tiket Bantuan

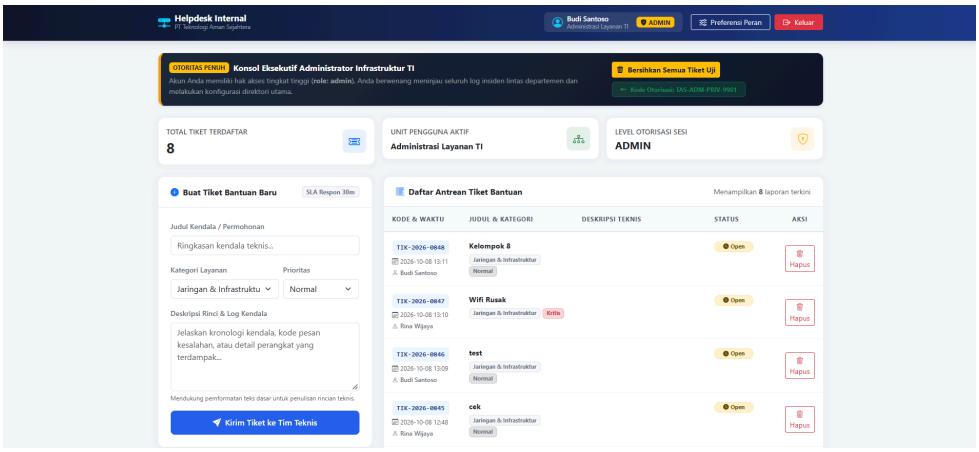
Gambar 2 Setelah tiket dikirim, tiket berhasil dibuat dan muncul pada Daftar Antrean Tiket Bantuan. Hal ini menunjukkan
bahwa input yang diberikan telah diterima dan disimpan oleh aplikasi. Pada tahap ini, payload XSS menjadi Stored
XSS karena data berbahaya tidak hanya diproses sementara, tetapi tersimpan di dalam

sistem dan akan dipanggil
kembali ketika halaman tiket ditampilkan. Pada dokumen, tiket yang digunakan adalah TIK-2026-0844 dengan judul “Wifi Rusak”



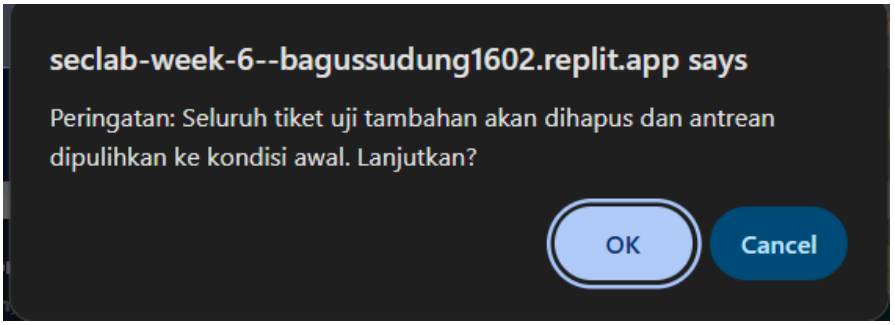
Gambar 3.Notification Allert XSS

Pada gambar 3 terlihat pop-up alert(1) yang muncul ketika halaman tiket dirender oleh browser. Pop-up tersebut membuktikan bahwa payload JavaScript berhasil melewati sanitasi dan dieksekusi oleh browser. Hal ini terjadi karena filter hanya menghapus string <script> satu kali. Setelah bagian <script> di tengah payload dihapus, bagian <scr dan ipt> yang tersisa menyatu kembali sehingga membentuk tag <script> yang utuh. Hasil akhirnya menjadi <script>alert(1)</script>.



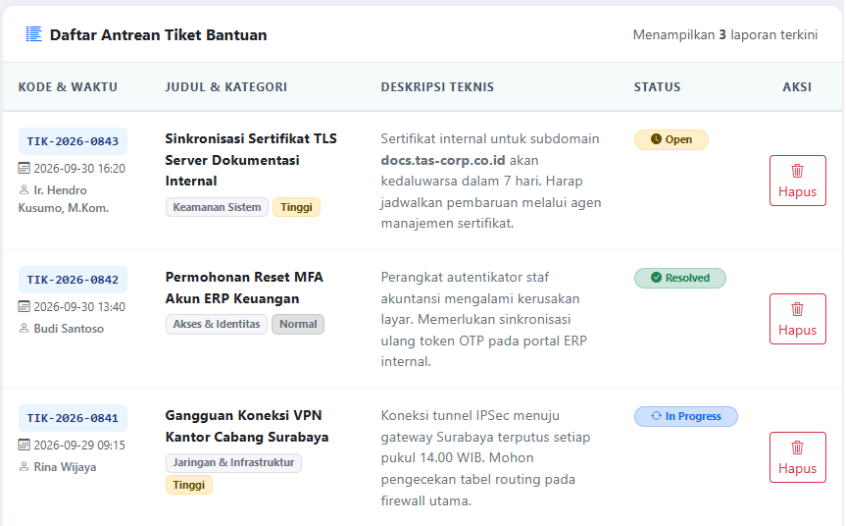
Gambar 4.Login Ke akun Admin untuk melakukan pembersihan Semua Tiket Bantuan

Pada tahap berikutnya, pengujian dilakukan menggunakan akun Admin untuk melihat dampak dari Stored XSS terhadap pengguna dengan hak akses tinggi. Akun Admin memiliki wewenang yang lebih besar dibandingkan pengguna biasa. Oleh karena itu, apabila tiket yang mengandung Stored XSS dibuka oleh Admin, JavaScript yang tersimpan dapat berjalan dalam konteks sesi browser Admin. Kondisi tersebut menjadi bagian penting dalam proses vulnerability chaining, yaitu menggabungkan Stored XSS dengan kelemahan CSRF pada endpoint perubahan hak akses.



Gambar 5.Notification Hapus daftar Antrean Bantuan

Gambar 5 Pada tahap ini, Stored XSS digunakan untuk memicu request ke endpoint /api/update-role. Endpoint tersebut diketahui menerima request POST untuk mengubah hak akses pengguna menjadi Admin, tetapi tidak memiliki perlindungan CSRF Token. Dengan demikian, ketika tiket berisi payload XSS dibuka oleh Admin, JavaScript yang berjalan dalam sesi browser tersebut dapat mengirim request perubahan role menggunakan konteks autentikasi korban. Inilah yang menunjukkan terjadinya vulnerability chaining dari Stored XSS menuju CSRF dan privilege escalation.

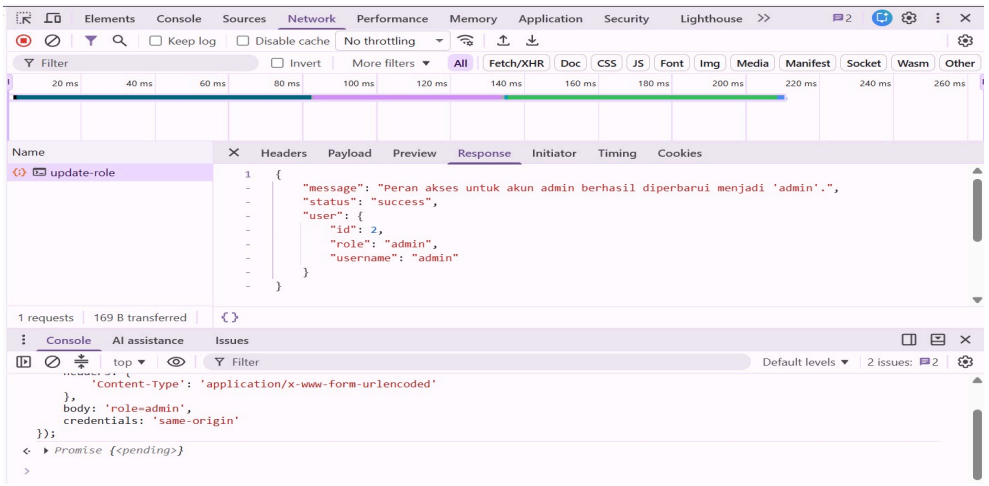


Gambar 6.Hasil nya akan seperti pada gambar ini

Gambar 6 pengujian menunjukkan bahwa filter XSS yang menggunakan metode blacklist dengan single-pass replacement dapat dilewati menggunakan teknik Tag Nesting. Payload berhasil tersimpan dan kemudian dieksekusi ketika halaman tiket dibuka, sehingga menghasilkan Stored XSS. Kerentanan tersebut kemudian dapat dirangkai dengan endpoint perubahan role yang tidak menggunakan CSRF Token, sehingga Stored XSS dapat digunakan untuk melakukan aksi sensitif dalam konteks sesi Admin. Rangkaian dua kerentanan tersebut disebut Vulnerability Chaining dan dapat meningkatkan dampak dari sekadar eksekusi JavaScript menjadi Privilege Escalation/Account Takeover.

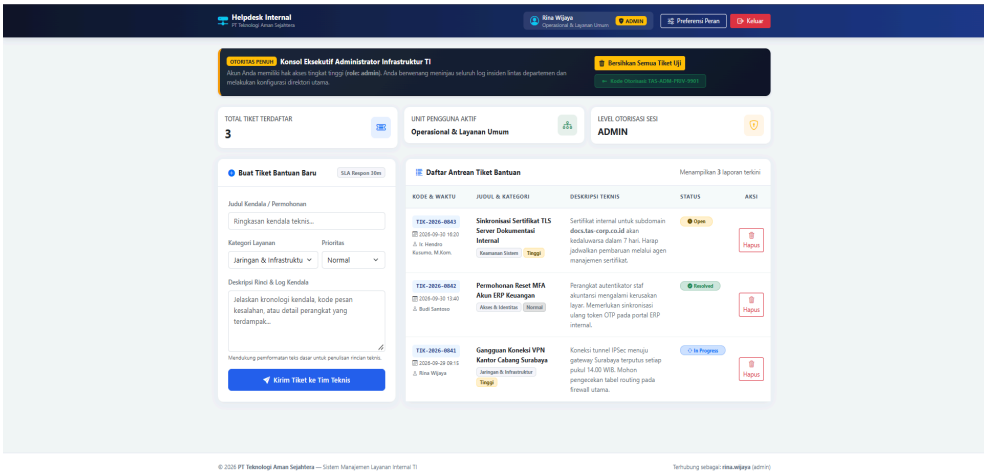
2. **The Kill Chain:** *Endpoint* `/api/update-role` diketahui rentan karena menerima *request* POST perubahan hak akses (parameter `role=admin`) tanpa perlindungan CSRF Token. Rancang ulang *payload Stored XSS* Anda menggunakan perintah `fetch()` pada JavaScript untuk memaksa browser milik Admin yang sedang mengecek tiket Anda agar secara diam-diam mengirimkan *request* POST ke *endpoint* tersebut.

Jawaban :



Gambar 1.menunjukan Hasil Request Api/Update-role

Pada gambar 1 terlihat hasil pengujian terhadap endpoint /api/update-role. Request yang dikirim menggunakan metode POST untuk melakukan perubahan hak akses pengguna. Endpoint tersebut tidak menggunakan CSRF Token, sehingga request dapat diproses tanpa adanya token validasi tambahan. Kondisi ini menunjukkan bahwa endpoint perubahan role memiliki kelemahan terhadap serangan CSRF dan dapat dimanfaatkan sebagai bagian dari vulnerability chaining dengan Stored XSS.



Gambar 2.Ini menunjukan bahwa berhasil mengganti Role STAFF menjadi Role Admin

Pada gambar 2 terlihat bahwa proses pengujian berhasil menyebabkan akun dengan

role Staff berubah menjadi Admin. Perubahan tersebut menunjukkan bahwa request perubahan hak akses berhasil diproses oleh sistem. Dalam skenario vulnerability chaining, Stored XSS yang berjalan pada browser Admin digunakan untuk mengirim request ke endpoint /api/update-role menggunakan konteks sesi pengguna tersebut. Karena endpoint tidak memiliki perlindungan CSRF Token, request perubahan role dapat diterima oleh server.

3. Analisis Rantai: Jelaskan konsep Vulnerability Chaining yang baru saja Anda lakukan.Mengapa Stored XSS adalah senjata paling mematikan untuk mengeksploitasi kerentanan Auth Flaw atau CSRF?

Jawaban :

Konsep Vulnerability Chaining : Penggabungan dua kerentanan (Stored XSS dan ketiadaan Anti-CSRF Token) untuk meningkatkan dampak serangan dari eksploitasi lokal menjadi penataan ulang hak akses secara menyeluruh (Privilege Escalation / Account Takeover).

Mengapa Stored XSS Sangat Mematikan : Stored XSS berjalan langsung di dalam konteks sesi browser milik korban. Ketika dieksekusi pada akun dengan wewenang tinggi (Admin), skrip dapat melakukan aksi sensitif atas nama korban tanpa memerlukan konfirmasi interaktif, melewati mekanisme kontrol akses berbasis IP, dan memanfaatkan kredensial sesi yang sedang aktif.

BAGIAN 3: REMEDIASI DAN SECURE CODING (Bobot: 30%) Seorang DevSecOps tidak hanya mengeksploitasi, tetapi juga menambal (*patch*) infrastruktur.

1. **Patch SQL Injection:** Tuliskan ulang *snippet* kode eksekusi *database* pada Bagian 1 menggunakan prinsip *Prepared Statements (Parameterized Queries)* agar aman dari SQLi secara absolut, sehingga fungsi `replace()` tidak lagi dibutuhkan.

Jawaban:

```
username = request.form['username']
password = request.form['password']

query = "SELECT * FROM users WHERE username = %s"
cursor.execute(query, (username,))
user = cursor.fetchone()

if user and verify_password(password, user['password_hash']):
    # login berhasil
    ...
else:
    # login gagal
    …
```

2. **Patch XSS:** Jelaskan mengapa pendekatan "Blacklist" (*replace string* spesifik) adalah praktik buruk. Sebutkan nama metode atau fungsi *built-in* yang seharusnya digunakan *developer* untuk membersihkan input HTML sebelum direnderke DOM (misalnya: *HTML Entity Encoding*).

Jawaban:

Pendekatan Blacklist yang diterapkan developer, yaitu menghapus string spesifik seperti script, onload, onerror, dan javascript: menggunakan fungsi replace(), merupakan praktik buruk karena beberapa alasan teknis. Blacklist bersifat tidak lengkap secara inheren karena hanya menutup pola yang sudah diketahui, padahal HTML5 memiliki banyak event handler lain yang tidak masuk daftar seperti onfocus dan onpointerenter, serta vektor non script seperti svg dengan onload atau img dengan onerror. Blacklist juga rentan terhadap bypass struktural akibat single pass replacement, seperti terbukti pada eksploitasi Bagian 2, di mana filter yang menghapus string script hanya dieksekusi satu kali sehingga gagal menangani teknik tag nesting scr script ipt, karena sisa karakter scr dan ipt menyatu kembali membentuk tag script yang utuh setelah bagian tengahnya dihapus. Selain itu blacklist tidak context aware karena filter yang sama diterapkan pada seluruh output tanpa mempertimbangkan konteks rendering, padahal karakter yang aman pada HTML body belum tentu aman pada atribut HTML maupun inline JavaScript. Secara prinsip, blacklist menganut pendekatan deny known bad yang reaktif, bukan allow known good yang proaktif, padahal prinsip keamanan yang benar mengarah pada pendekatan kedua.

Metode yang seharusnya digunakan developer adalah HTML Entity Encoding atau Output Encoding, yaitu mengonversi karakter yang memiliki makna khusus dalam HTML menjadi entitas sehingga browser merendernya sebagai teks biasa, bukan markup yang dieksekusi, seperti kurung sudut buka menjadi lt, kurung sudut tutup menjadi gt, ampersand menjadi amp, tanda kutip ganda menjadi quot, dan tanda kutip tunggal menjadi x27. Pada Python tersedia fungsi html.escape(), pada Flask dengan Jinja2 auto escaping sudah aktif default atau dapat dipanggil manual lewat markupsafe.escape(), pada PHP tersedia htmlspecialchars(), dan pada JavaScript dapat digunakan escape html atau DOMPurify untuk kasus yang membutuhkan sebagian tag HTML tetap diizinkan. Encoding dilakukan pada titik output saat data direnderke DOM, bukan pada titik input, sehingga data mentah tetap utuh namun selalu aman ditampilkan apa pun isinya. Sebagai lapisan tambahan, developer disarankan menerapkan header Content Security Policy dengan konfigurasi script src self agar browser menolak mengeksekusi inline script meskipun ada payload yang lolos dari proses encoding.

3. **Patch CSRF:** Langkah teknis apa yang harus ditambahkan *developer* pada arsitektur *form/API* mereka untuk mengamankan *endpoint* `/api/update-role` dari pemalsuan *request* lintas situs?

Jawaban:

Langkah teknis utama adalah menerapkan Anti CSRF Token dengan Synchronizer Token Pattern pada endpoint /api/update-role. Server men-generate token acak menggunakan cryptographically secure random generator seperti secrets.token_urlsafe(32), menyimpannya terikat pada session di server, lalu mengirimkannya ke client sebagai hidden field atau custom

header seperti X-CSRF-Token. Setiap request POST ke endpoint tersebut wajib menyertakan token ini, dan server memvalidasi kecocokannya dengan token di session sebelum memproses request, menolak dengan status 403 jika tidak valid. Mekanisme ini menutup celah pada Bagian 2 karena payload Stored XSS yang mengirim request lewat fetch() tetap otomatis membawa session cookie Admin, namun tidak memiliki token CSRF yang valid.

Sebagai lapisan tambahan, cookie session perlu diberi atribut SameSite Strict atau Lax agar tidak ikut terkirim pada request lintas origin, serta atribut HttpOnly agar tidak bisa dibaca lewat JavaScript. Validasi header Origin atau Referer di server juga dapat ditambahkan sebagai pengecekan sekunder. Khusus endpoint sensitif seperti update-role, disarankan menambahkan step up authentication berupa verifikasi ulang password sebelum perubahan role diproses.

D. FORMAT LAPORAN DAN PENGUMPULAN Laporan disusun dalam format **Penetration Testing & Remediation Report** bergaya eksekutif korporat yang memuat:

1. **Halaman Sampul:** Nama Kelompok, Daftar Nama Anggota, Nama Divisi, Nama Program.
2. **Executive Summary:** Ringkasan insiden dan kerentanan yang ditemukan. 3. **Proof of Concept (PoC):** Langkah-langkah detail perakitan *payload evasion*, *vulnerability chaining*, dan *screenshot* eksekusi (menjawab Bagian 1 & 2). 4. **Remediation & Secure Coding Patch:** Usulan arsitektur pertahanan dan *snippet* kode perbaikan (menjawab Bagian 3).
5. **Format Penamaan Berkas:** Ekspor ke PDF dengan nama: `[Nama Kelompok]_[Nama Divisi]_Tugas Modul 6.pdf`.

Total Nilai Maksimal: 100 poin.